

# Improving Turkish Legal Question Answering with an Optimized RAG Pipeline

**Course:** CENG493 - Term Project

**Project Type:** Legal Question Answering / Retrieval-Augmented Generation

**Instructor:** Assoc. Prof. Serdar ARSLAN

**Team Members:** Deniz Arda Çınarer (202211019), İbrahim Ersan Özdemir (202211054)

**Repository:** <https://github.com/SpeedyV5/turkish-legal-rag>

**Final Submission Date:** May 2026

## Abstract

This project develops a domain-adapted Retrieval-Augmented Generation (RAG) system for Turkish legal question answering. The system takes a Turkish legal question as input and returns a grounded, source-supported answer with legal article citations. The final pipeline combines article-aware legal document chunking, multilingual dense retrieval, cross-encoder reranking, retrieval-aware prompting, and QLoRA-based supervised fine-tuning of a local instruction-tuned LLM.

The final evaluation follows the strongest rubric scenario, **Gold Q + A + Doc**, because the project includes a gold benchmark with questions, reference answers, and relevant legal document/article labels. Retrieval is evaluated with MRR, Recall@k, nDCG, Hit@k, and article-level recall. Answer quality is evaluated with Exact Match, token-level F1, BLEU-1, BLEU-2, and ROUGE-L F1. Grounding is evaluated with citation metrics and a lexical faithfulness proxy. The main comparison uses the same base LLM, `Qwen/Qwen2.5-3B-Instruct`, for Base RAG and Fine-tuned RAG, where the fine-tuned system uses a QLoRA adapter trained on project-specific legal QA examples.

On the held-out test split, the Fine-tuned RAG system improves Answer F1 from **0.2567** to **0.4031**, Citation Exact Match from **0.0968** to **0.4516**, and lexical faithfulness from **0.7454** to **0.9041** compared with Base RAG using the same retrieval stack. The final system also supports evaluator-provided custom PDF collections and evaluator-provided benchmark files, allowing the same evaluation pipeline to be rerun on new data.

## Table of Contents

1. Introduction
2. Assignment and Rubric Alignment
3. Dataset and Corpus
4. System Architecture
5. Benchmark and Evaluation Methodology
6. Retrieval Experiments
7. Base RAG vs Fine-tuned RAG
8. Ablation Study
9. Fine-tuning Configuration
10. Error and Hallucination Analysis

- 11. Custom Corpus and Custom Benchmark Support
- 12. Reproducibility and Hardware
- 13. Limitations and Future Work
- 14. Final Conclusion

## 1. Introduction

Turkish legal question answering requires more than fluent natural language generation. A useful legal QA system must retrieve the correct legal sources, answer only from those sources, and cite the relevant legal articles consistently. A generic LLM may produce plausible but unsupported legal statements, which is risky in a legal domain. Therefore, this project uses Retrieval-Augmented Generation (RAG) to connect answer generation to an explicit legal corpus.

The project objective is to improve Turkish legal QA with optimization in three major RAG components:

- **Retrieval:** embedding model selection, dense retrieval, BM25/hybrid retrieval experiments, and FAISS indexing.
- **Reranking:** multilingual cross-encoder reranking, including comparison with English-only reranking.
- **Generation:** retrieval-aware prompting and QLoRA supervised fine-tuning of a local instruction-tuned LLM.

The final deliverable is a locally runnable legal RAG system with a measurable benchmark, documented experiments, a Base RAG vs Fine-tuned RAG comparison, an ablation study, error analysis, and live demo support.

## 2. Assignment and Rubric Alignment

The term project assignment requires a domain-adapted RAG system for Turkish legal QA. The input is a Turkish legal question, and the output must be a grounded, context-aware answer with citation consistency. The project also requires a gold QA benchmark of 150-300 questions, reproducible experiments, documented hyperparameters, GPU usage reporting, a technical report, a GitHub repository, a presentation, a live demo, and error analysis.

The evaluation rubric defines three possible scenarios. This project matches **Scenario 1: Gold Q + A + Doc**, because it contains:

- Gold questions.
- Gold reference answers.
- Gold relevant document/article labels.

Under this scenario, the rubric expects retrieval evaluation, answer quality evaluation, and grounding evaluation. The project covers these as follows:

| Rubric area           | Required by rubric | Project implementation                                                      |
|-----------------------|--------------------|-----------------------------------------------------------------------------|
| Retrieval             | Recall@k, MRR      | MRR, Recall@1/3/5/10, Precision@k, nDCG@k, Hit@k, article-level recall      |
| Answer                | EM/F1 or LLM Judge | Exact Match, token-level Answer F1, precision, recall, BLEU-1/2, ROUGE-L F1 |
| Grounding             | Faithfulness       | Citation precision/recall/F1/exact, Has Dayanak, lexical faithfulness proxy |
| End-to-end evaluation | Full pipeline      | Retrieval -> reranking -> generation -> metrics on the same benchmark       |

The rubric screenshot also states a weighted Scenario 1 scoring focus:

$$\text{Final} = 0.35R + 0.4A + 0.25G$$

For transparent reporting, this project defines a rubric-compatible score where:

- **R** = average of MRR, Recall@5, and Recall@10.
- **A** = Answer F1.
- **G** = lexical faithfulness proxy.

Using the held-out test comparison:

| System         | R      | A      | G      | Rubric-compatible score |
|----------------|--------|--------|--------|-------------------------|
| Base RAG       | 0.7918 | 0.2567 | 0.7454 | 0.5662                  |
| Fine-tuned RAG | 0.7918 | 0.4031 | 0.9041 | 0.6643                  |

This composite is not presented as the only score, but it aligns the reported results with the evaluation form. The main detailed metrics are still reported separately because legal QA quality is multi-dimensional.

## 3. Dataset and Corpus

### 3.1 Legal Document Corpus

The base corpus contains 7 fundamental Turkish legal documents:

| Document                               | Law No. |
|----------------------------------------|---------|
| Constitution of the Republic of Turkey | 2709    |
| Turkish Penal Code                     | 5237    |
| Criminal Procedure Code                | 5271    |
| Turkish Civil Code                     | 4721    |
| Turkish Code of Obligations            | 6098    |
| Civil Procedure Code                   | 6100    |
| Administrative Procedure Law           | 2577    |

The corpus pipeline downloads or registers legal PDFs, extracts text, creates a registry, chunks the legal text, embeds chunks, and builds a FAISS vector index. The chunking strategy is article-aware: it first preserves legal article boundaries and then splits long articles into manageable sub-chunks. This design is important because Turkish legislation is naturally article-based, and citation quality depends on preserving article references.

### 3.2 Gold Benchmark

The project includes a gold benchmark dataset with 175 Turkish legal QA examples. This satisfies the assignment requirement of a 150-300 question gold test set.

| Property          | Value                                                               |
|-------------------|---------------------------------------------------------------------|
| Total questions   | 175                                                                 |
| Train split       | 112                                                                 |
| Dev split         | 32                                                                  |
| Test split        | 31                                                                  |
| Gold fields       | question, expected answer, relevant document IDs, relevant articles |
| Question types    | definition, list, factual, procedural, yes/no                       |
| Difficulty levels | easy, medium, hard                                                  |

Benchmark files:

| File                                      | Purpose                   |
|-------------------------------------------|---------------------------|
| data/benchmark/gold_benchmark.jsonl       | Full benchmark            |
| data/benchmark/gold_benchmark_train.jsonl | Training split            |
| data/benchmark/gold_benchmark_dev.jsonl   | Development split         |
| data/benchmark/gold_benchmark_test.jsonl  | Held-out final test split |
| data/sft/sft_train.jsonl                  | QLoRA SFT training data   |

The train/dev/test split is used to avoid leakage. The QLoRA training data is derived only from the training split, while the final comparison is reported on the held-out test split.

### 3.3 Fine-tuning Data Considerations

The assignment lists external Turkish legal fine-tuning corpora such as Kaggle Turkish legal datasets and HuggingFace Turkish law chatbot data. These sources were reviewed during the project. For the final QLoRA experiment, the project prioritized its own verified gold benchmark training split because it directly matches the legal corpus and citation format. This reduces the risk of style mismatch and benchmark leakage.

## 4. System Architecture

The baseline RAG architecture is:

Question -> Embedding -> Vector Search -> Optional Reranker -> LLM -> Answer

The final project architecture is:

```
Turkish question
-> multilingual-e5-large embedding
-> FAISS dense retrieval
-> BGE cross-encoder reranking
-> retrieval-aware prompt construction
-> Qwen2.5-3B-Instruct + QLoRA adapter
-> answer + Dayanak citation
```

The final interactive command is:

```
python -m src.pipeline.rag_pipeline --lora-adapter outputs/sft_qlora/final
```

For live presentation, a safer short-answer mode is provided:

```
python -m src.pipeline.rag_pipeline --lora-adapter outputs/sft_qlora/final --demo-safe
```

The `--demo-safe` flag is a presentation mode. It keeps the final retrieval stack but asks the generator for shorter answers and appends the final `Dayanak:` citation from retrieved sources. Benchmark results are reported from the normal evaluation pipeline, not from demo-safe display formatting.

## 5. Benchmark and Evaluation Methodology

The project evaluates the system end-to-end. Retrieval metrics measure whether the system finds the correct legal source. QA metrics measure whether the generated answer matches the reference answer. Grounding metrics measure whether the answer is supported by retrieved context and whether citations match gold articles.

### 5.1 Retrieval Metrics

| Metric               | Meaning                                                       |
|----------------------|---------------------------------------------------------------|
| MRR                  | Rank quality of the first correct result                      |
| Recall@k             | Fraction of gold relevant articles retrieved in top-k         |
| Precision@k          | Fraction of retrieved top-k items that are relevant           |
| nDCG@k               | Ranking quality with more credit for earlier relevant results |
| Hit@k                | Whether at least one relevant result appears in top-k         |
| Article-level recall | Matching at legal article level rather than only chunk level  |

### 5.2 Answer Metrics

| Metric                  | Meaning                                               |
|-------------------------|-------------------------------------------------------|
| Exact Match             | Normalized exact match against the reference answer   |
| Token-level F1          | Overlap between generated and reference answer tokens |
| Answer precision/recall | Token overlap precision and recall                    |
| BLEU-1 / BLEU-2         | Supplemental n-gram overlap                           |
| ROUGE-L F1              | Supplemental longest-common-subsequence overlap       |

BLEU and ROUGE are included to match assignment expectations, but they are treated as supporting signals rather than the main legal QA score. Legal answers can be correct even when wording differs from the reference.

## 5.3 Grounding and Citation Metrics

| Metric                     | Meaning                                                   |
|----------------------------|-----------------------------------------------------------|
| Citation precision         | How many cited articles are gold-relevant                 |
| Citation recall            | How many gold articles are cited                          |
| Citation F1                | Harmonic mean of citation precision and recall            |
| Citation exact match       | Whether the citation set exactly matches gold articles    |
| Has Dayanak                | Whether the answer includes a citation line               |
| Faithfulness lexical proxy | Whether answer content tokens appear in retrieved context |

The faithfulness metric is lexical, not an LLM-as-judge score. This is explicitly documented as a limitation. It is still useful as a reproducible proxy for hallucination risk.

## 6. Retrieval Experiments

The project evaluates 10 retrieval variants. The strongest final retrieval system is `e5large_reranked_bge`.

| System                       | MRR    | Recall@5 | Recall@10 |
|------------------------------|--------|----------|-----------|
| baseline_dense (e5-base)     | 0.5896 | 0.7429   | 0.7933    |
| bm25_only                    | 0.3208 | 0.4657   | 0.5667    |
| hybrid (e5-base + BM25)      | 0.5510 | 0.7171   | 0.8057    |
| dense_reranked (English CE)  | 0.4139 | 0.6229   | 0.7286    |
| hybrid_reranked (English CE) | 0.4122 | 0.5943   | 0.7257    |
| dense_reranked_ml            | 0.6568 | 0.7524   | 0.8086    |
| hybrid_reranked_ml           | 0.6587 | 0.7533   | 0.8143    |
| e5large_dense                | 0.6773 | 0.7629   | 0.8476    |
| e5large_reranked_ml          | 0.6644 | 0.7952   | 0.8514    |
| e5large_reranked_bge         | 0.6964 | 0.8048   | 0.8743    |

Key findings:

- BM25 alone is weak because Turkish morphology reduces exact lexical matching reliability.
- Hybrid retrieval improves some recall cases but can add noise to top-ranked results.
- English-only cross-encoder reranking is harmful for Turkish legal text.
- The larger multilingual E5 embedding model improves semantic retrieval.
- BGE reranking gives the best final MRR, Recall@5, and Recall@10.

## 7. Base RAG vs Fine-tuned RAG

The professor specifically requires Base RAG and Fine-tuned RAG comparison using the same LLM. This project compares the same base model, `Qwen/Qwen2.5-3B-Instruct`, with and without a QLoRA adapter. Both systems use the same final retrieval stack in the held-out test comparison.

| System         | Retrieval               | LLM                             |
|----------------|-------------------------|---------------------------------|
| Base RAG       | e5-large + BGE reranker | Untuned Qwen2.5-3B-Instruct     |
| Fine-tuned RAG | e5-large + BGE reranker | QLoRA-tuned Qwen2.5-3B-Instruct |

Held-out test split results:

| Metric                     | Base RAG | Fine-tuned RAG | Delta   |
|----------------------------|----------|----------------|---------|
| Answer F1                  | 0.2567   | 0.4031         | +0.1464 |
| Answer Precision           | 0.2273   | 0.4051         | +0.1778 |
| Answer Recall              | 0.3622   | 0.4598         | +0.0976 |
| Citation F1                | 0.4851   | 0.5742         | +0.0891 |
| Citation Precision         | 0.3946   | 0.5806         | +0.1860 |
| Citation Recall            | 0.8065   | 0.6237         | -0.1828 |
| Citation Exact Match       | 0.0968   | 0.4516         | +0.3548 |
| Has Dayanak                | 0.8387   | 0.9677         | +0.1290 |
| Faithfulness lexical proxy | 0.7454   | 0.9041         | +0.1587 |

The fine-tuned model improves answer quality, citation exactness, and faithfulness. Citation recall decreases, but citation precision and exact match improve substantially. In a legal QA context, this is a reasonable trade-off because fewer but more accurate citations are preferable to many loosely related citations.

Supplemental overlap metrics:

| Metric     | Base RAG | Fine-tuned RAG | Delta   |
|------------|----------|----------------|---------|
| BLEU-1     | 0.2387   | 0.3460         | +0.1073 |
| BLEU-2     | 0.1869   | 0.3063         | +0.1194 |
| ROUGE-L F1 | 0.2718   | 0.4083         | +0.1365 |

## 8. Ablation Study

The assignment asks for ablation experiments comparing baseline RAG, embedding improvement, reranker contribution, LLM fine-tuning, and the fully optimized system. The project reports this as a four-step practical ablation because embedding and reranker fine-tuning were not feasible on local GPU resources. The embedding step is therefore documented honestly as **model selection**, not embedding fine-tuning.

| Variant                        | Retrieval      | LLM             | MRR    | Recall@5 | Answer F1 | Citation F1 | Citation Exact | Faithfulness |
|--------------------------------|----------------|-----------------|--------|----------|-----------|-------------|----------------|--------------|
| 1. Baseline RAG                | e5-base dense  | Untuned Qwen 3B | 0.5896 | 0.7429   | 0.2453    | 0.3765      | 0.1290         | 0.7228       |
| 2. + Embedding model selection | e5-large dense | Untuned Qwen 3B | 0.6773 | 0.7629   | 0.3024    | 0.5233      | 0.1935         | 0.7615       |
| 3. + Reranker                  | e5-large + BGE | Untuned Qwen 3B | 0.6964 | 0.8048   | 0.2567    | 0.4851      | 0.0968         | 0.7454       |
| 4. + LLM fine-tuning           | e5-large + BGE | QLoRA Qwen 3B   | 0.6964 | 0.8048   | 0.4031    | 0.5742      | 0.4516         | 0.9041       |

Interpretation:

- The embedding upgrade improves retrieval MRR and recall.
- BGE reranking further improves retrieval quality.
- Better retrieval alone does not guarantee better answer generation; the untuned generator can still fail to use context optimally.
- QLoRA SFT provides the largest generation-side improvement.

Not completed:

- **Embedding fine-tuning:** Not performed due to local VRAM and time constraints.
- **Reranker fine-tuning:** Not performed because zero-shot BGE reranking already performed strongly and fine-tuning would require a separate query-positive-negative training set.

These limitations are documented rather than hidden, which is important for a reproducible academic report.

## 9. Fine-tuning Configuration

---

The generator is fine-tuned with QLoRA supervised fine-tuning.



| Parameter             | Value                                |
|-----------------------|--------------------------------------|
| Base model            | Qwen/Qwen2.5-3B-Instruct             |
| Quantization          | 4-bit NF4                            |
| LoRA rank             | 16                                   |
| LoRA alpha            | 32                                   |
| LoRA dropout          | 0.05                                 |
| Target modules        | q_proj, k_proj, v_proj, o_proj       |
| Epochs                | 3                                    |
| Batch size            | 1                                    |
| Gradient accumulation | 8                                    |
| Learning rate         | 2e-4                                 |
| Scheduler             | cosine                               |
| Max sequence length   | 1024                                 |
| Training examples     | 112                                  |
| Training hardware     | NVIDIA RTX 3070 Laptop GPU, 8GB VRAM |
| Training time         | Approximately 22 minutes             |

The training examples follow a retrieval-aware supervised format:

```
System: Turkish legal assistant instruction
User: Question + retrieved legal context
Assistant: Gold answer + Dayanak citation
```

This teaches the model not only to answer Turkish legal questions, but also to follow the project-specific citation format.

## 10. Error and Hallucination Analysis

Hallucination analysis is mandatory in the assignment. This project evaluates hallucination risk through lexical faithfulness, missing citation detection, citation mismatch, and per-question error analysis.

Held-out test summary:

| Error signal             | Base RAG | Fine-tuned RAG |
|--------------------------|----------|----------------|
| Good answers (F1 > 0.5)  | 4/31     | 13/31          |
| Low answer F1 (< 0.25)   | 17/31    | 11/31          |
| Low faithfulness (< 0.5) | 5/31     | 2/31           |
| Missing Dayanak          | 5/31     | 1/31           |
| Citation exact mismatch  | 28/31    | 17/31          |

Remaining risk categories:

- **Penalty and qualified-offense questions:** Turkish Penal Code questions involving exact penalty amounts and qualified forms remain challenging.
- **Procedural questions:** Civil procedure and administrative procedure questions can be faithful but incomplete.
- **Citation recall trade-off:** Fine-tuning improves citation exactness but reduces citation recall.
- **Generator limitation:** The 3B local LLM is the main bottleneck for complex legal reasoning.

The most important conclusion is that retrieval is relatively strong, but answer generation remains the highest-risk component. The final demo mode is therefore designed to prefer concise, grounded answers over long explanations.

## 11. Custom Corpus and Custom Benchmark Support

---

### 11.1 Custom PDF Corpus

The professor must be able to provide a custom document collection. This project supports evaluator-provided PDF folders with `scripts/prepare_custom_pdfs.py`.

Workflow:

```
python scripts/prepare_custom_pdfs.py --input-dir path/to/custom_pdfs --reset
python -m src.corpus.build_registry
python -m src.corpus.register_pdfs
python -m src.retrieval.chunking
python -m src.retrieval.embedder --config configs/retrieval_config_e5large.yaml
python -m src.retrieval.vector_store --config configs/retrieval_config_e5large.yaml
python -m src.pipeline.rag_pipeline --lora-adapter outputs/sft_qlora/final --demo-safe
```

This rebuilds the local corpus artifacts and FAISS index from the evaluator-provided PDFs.

### 11.2 Custom Benchmark

The professor may also provide a custom benchmark file. The QA evaluation script accepts an external JSONL benchmark through `--benchmark`.

Example:

```
python -m src.evaluation.run_qa_eval --benchmark path/to/custom_benchmark.jsonl --system
e5large_reranked_bge --lora-adapter outputs/sft_qlora/final --output-tag custom
```

Expected JSONL schema:

```
{
  "id": "custom_001",
  "question": "Question text",
  "expected_answer": "Reference answer",
  "relevant_doc_ids": ["doc_id"],
  "relevant_articles": ["Madde 1"],
  "question_type": "definition",
  "difficulty": "easy",
  "source_law": "Custom Document"
}
```

This directly addresses the requirement that systems can be evaluated on a professor-prepared benchmark using the same metrics.

## 12. Reproducibility and Hardware

### 12.1 Main Commands

Install dependencies:

```
python -m pip install -r requirements.txt
```

Run final interactive system:

```
python -m src.pipeline.rag_pipeline --lora-adapter outputs/sft_qlora/final
```

Run live demo-safe mode:

```
python -m src.pipeline.rag_pipeline --lora-adapter outputs/sft_qlora/final --demo-safe
```

Run final QA evaluation:

```
python -m src.evaluation.run_qa_eval --split test --system e5large_reranked_bge --lora-adapter outputs/sft_qlora/final --output-tag sft_qlora
```

### 12.2 Environment

| Item                   | Value                                                                   |
|------------------------|-------------------------------------------------------------------------|
| Python                 | 3.11                                                                    |
| CUDA                   | 12.4                                                                    |
| PyTorch                | 2.7.0+cu124                                                             |
| Phase 2 GPU            | NVIDIA RTX 3050 Laptop GPU, 4GB VRAM                                    |
| Phase 3 GPU            | NVIDIA RTX 3070 Laptop GPU, 8GB VRAM                                    |
| Phase 4 local demo GPU | NVIDIA RTX 3050 Laptop GPU, 4GB VRAM                                    |
| Main libraries         | transformers, sentence-transformers, peft, trl, bitsandbytes, faiss-cpu |

Observed benchmark latency is approximately 17-21 seconds per generated answer, while full smoke-test examples can take approximately 26-31 seconds depending on model loading, reranking, and GPU memory state.

## 13. Limitations and Future Work

Limitations:

- Embedding fine-tuning was not performed.
- Reranker fine-tuning was not performed.

- Faithfulness is a lexical proxy, not an NLI or LLM-as-judge score.
- The corpus is limited to 7 legislation PDFs.
- The final generator is a small local 3B model and can struggle with complex multi-condition legal questions.
- Live demo-safe mode is optimized for presentation stability; formal benchmark results come from the normal evaluation pipeline.

Future work:

- Contrastive embedding fine-tuning with hard negative mining.
- BGE reranker fine-tuning on Turkish legal relevance pairs.
- Larger LLM or API-backed model comparison.
- LLM-as-judge or NLI-based faithfulness scoring.
- Expansion to Yargitay decisions, TBMM records, and broader legal sources.
- Larger held-out benchmark or k-fold cross-validation.

## 14. Final Deliverables

The final submission package contains:

| Deliverable              | Status                                        |
|--------------------------|-----------------------------------------------|
| GitHub repository        | Ready                                         |
| Technical report         | REPORT.md and PDF export                      |
| Gold benchmark           | data/benchmark/gold_benchmark*.jsonl          |
| Evaluation artifacts     | outputs/evaluation/ , outputs/qa_eval/        |
| Fine-tuned LoRA adapter  | outputs/sft_qlora/final/                      |
| Demo guide               | DEMO_GUIDE.md                                 |
| Custom corpus support    | scripts/prepare_custom_pdfs.py                |
| Custom benchmark support | src/evaluation/run_qa_eval.py --benchmark ... |

## 15. Final Conclusion

The final project is a complete, locally runnable Turkish legal RAG system with a documented gold benchmark, measurable retrieval and QA evaluation, a Base RAG vs Fine-tuned RAG comparison using the same LLM, an ablation study, hallucination/error analysis, custom document support, and custom benchmark support.

The final system's strongest result is the improvement from the untuned base generator to the QLoRA-tuned generator on the same final retrieval stack. Answer F1 improves from **0.2567** to **0.4031**, Citation Exact Match improves from **0.0968** to **0.4516**, and lexical faithfulness improves from **0.7454** to **0.9041**. These results show that fine-tuning improves not only answer overlap, but also citation discipline and groundedness.

The remaining bottleneck is generation quality from a small local 3B LLM on complex Turkish legal questions. Retrieval is already strong, with final MRR **0.6964**, Recall@5 **0.8048**, and Recall@10 **0.8743**. Future improvements should therefore focus on stronger generation, domain-specific reranker tuning, and stronger faithfulness evaluation.